



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 13

Team No.: 18

Project Title: Multithreaded Linux Application Using POSIX Threads and Mutexes

Team Members

Roll Number	Student Name	Signature
2520090022	K.Sriram	
2520090048	I.Haricharansai	
2520080041	Sathwik Naidu Sannapaneni	

1. Abstract

The project “**Multithreaded Linux Application Using POSIX Threads and Mutexes**” focuses on developing a Linux-based application capable of performing multiple tasks concurrently using POSIX threads (pthreads). In conventional single-threaded applications, tasks are executed sequentially, which can reduce performance and responsiveness when multiple operations need to be processed simultaneously. This project addresses this problem by implementing a multithreaded architecture in which multiple threads execute independent tasks concurrently while safely sharing common resources.

The project demonstrates important **Operating Systems and System Programming concepts**, including process and thread management, concurrency, synchronization, mutual exclusion, race conditions, critical sections, thread creation and termination, and shared-memory communication. **POSIX threads (pthread)** are used to create and manage threads, while **mutex locks (pthread_mutex)** are implemented to protect shared resources and prevent inconsistent data caused by simultaneous access.

The proposed solution will be developed and executed on a **Linux operating system** using the GCC compiler and POSIX APIs. The application will divide a computational or resource-management task among multiple threads, synchronize access to shared data using mutexes, and demonstrate the difference between synchronized and unsynchronized execution.

The expected outcome is a functional and reliable multithreaded Linux application that demonstrates efficient concurrent execution while maintaining data consistency. The project will provide practical understanding of how modern operating systems manage threads, synchronization, and shared resources.

2. Problem Statement

Single-threaded applications execute tasks sequentially, which can reduce performance and CPU utilization when multiple tasks need to run simultaneously. This project aims to develop a **Linux-based multithreaded application** that performs multiple tasks concurrently using **POSIX Threads (pthreads)**.

Since multiple threads may access shared resources at the same time, **race conditions and data inconsistency** can occur. The project uses **mutexes** to synchronize thread access and protect critical sections.

The project demonstrates key **Operating Systems and Systems Programming concepts**, including thread management, concurrency, synchronization, mutual exclusion, critical sections, and race-condition prevention.

3. Objectives

1. To develop a **multithreaded Linux application** using POSIX Threads (pthreads).
2. To implement **mutex-based synchronization** for safe access to shared resources.
3. To demonstrate **concurrency, critical sections, and race-condition prevention**.
4. To analyze the **performance and efficiency** of multithreaded execution.

4. Proposed Methodology

The project will be implemented as a **Linux-based C application** using the POSIX Threads (pthread) library. The application will divide the required tasks among multiple threads so that they can execute concurrently.

The major steps are:

1. **Initialize shared resources** and mutex locks.
2. **Create multiple threads** using `pthread_create()`.
3. Each thread performs its assigned task and accesses shared data when required.
4. **Mutex locks** are used before entering critical sections to prevent simultaneous access.
5. Threads release the mutex after completing the critical operation.
6. The main program waits for all threads using `pthread_join()`.
7. The results are collected and displayed after all threads complete execution.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
POSIX Threads (pthread)	Create and manage multiple threads.
pthread_create()	Creates new threads for concurrent execution.
pthread_join()	Waits for threads to complete execution.
Mutex (pthread_mutex)	Provides mutual exclusion for shared resources.
pthread_mutex_lock()	Locks a critical section before accessing shared data.
pthread_mutex_unlock()	Releases the lock after completing the critical operation.
Critical Section	Protects code that accesses shared resources.
Race Condition	Demonstrates problems caused by simultaneous resource access.
Concurrency	Allows multiple tasks to execute concurrently.
Shared Memory / Shared Variables	Allows threads to communicate and share data.
GCC Compiler	Compiles and builds the C program in Linux.
Linux Process/Thread Scheduling	Demonstrates how Linux manages and schedules threads.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating environment for developing and executing the application.
C / C++	Programming language used to implement multithreading and synchronization.
GCC Compiler	Compiles the C/C++ source code on Linux.
POSIX Threads (Pthreads)	Creates and manages multiple threads.
Mutex (pthread_mutex)	Synchronizes access to shared resources.
VS Code / Linux Terminal	Writing, compiling, running, and debugging the application.

8. Expected Outcome:

The project is expected to produce a **functional Linux-based multithreaded application** that executes multiple tasks concurrently using POSIX threads.

- Multiple threads will be created and executed successfully.
- Mutexes will prevent **race conditions and data inconsistency** when accessing shared resources.
- The application will display the **execution status and final results** of each thread.
- The project will demonstrate improved **concurrency, resource utilization, and synchronization** compared to single-threaded execution.
- It will provide practical understanding of **Linux thread management and Operating Systems synchronization mechanisms**.

9. GitHub Repository

GitHub Repository Link:

https://github.com/kunapareddysriram2007-cmyk/2520090022_OSSP
https://github.com/haricharansai/2520090048_OSSP
https://github.com/flare8809/2520080041-OSSP

Repository Created: Yes

Faculty Name and Signature

Faculty Name: Alekhya Ma'am

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____